

Cryptography and Network Security Chapter 11

Fifth Edition
by William Stallings

Lecture slides by Lawrie Brown

Lecture slides by Lawrie Brown for “Cryptography and Network Security”, 5/e, by William Stallings, Chapter 11 – “Cryptographic Hash Functions”.

Chapter 11 – Cryptographic Hash Functions

Each of the messages, like each one he had ever read of Stern's commands, began with a number and ended with a number or row of numbers. No efforts on the part of Mungo or any of his experts had been able to break Stern's code, nor was there any clue as to what the preliminary number and those ultimate numbers signified.

—*Talking to Strange Men*, Ruth Rendell

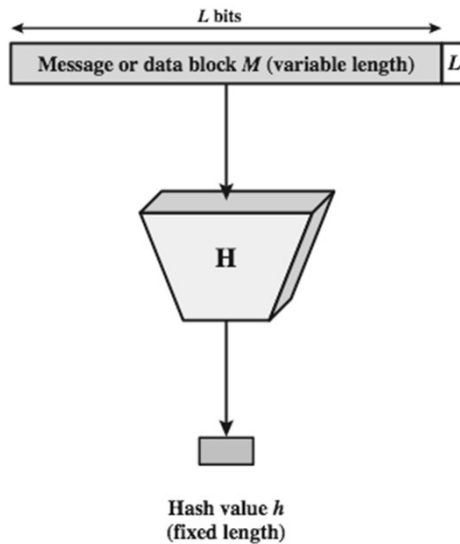
Opening quote.

Hash Functions

- condenses arbitrary message to fixed size
$$h = H(M)$$
- usually assume hash function is public
- hash used to detect changes to message
- want a cryptographic hash function
 - computationally infeasible to find data mapping to specific hash (one-way property)
 - computationally infeasible to find two data to same hash (collision-free property)

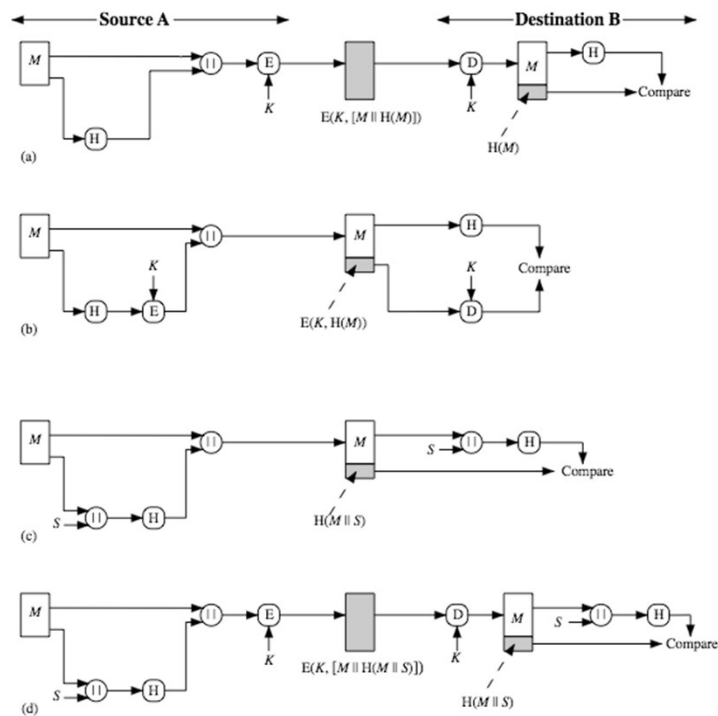
A hash function H accepts a variable-length block of data M as input and produces a fixed-size hash value $h = H(M)$. A "good" hash function has the property that the results of applying the function to a large set of inputs will produce outputs that are evenly distributed, and apparently random. In general terms, the principal object of a hash function is data integrity. A change to any bit or bits in M results, with high probability, in a change to the hash code. The kind of hash function needed for security applications is referred to as a cryptographic hash function. A cryptographic hash function is an algorithm for which it is computationally infeasible (because no attack is significantly more efficient than brute force) to find either (a) a data object that maps to a pre-specified hash result (the one-way property) or (b) two data objects that map to the same hash result (the collision-free property). Because of these characteristics, hash functions are often used to determine whether or not data has changed.

Cryptographic Hash Function



Stallings Figure 11.1 depicts the general operation of a cryptographic hash function. Typically, the input is padded out to an integer multiple of some fixed length (e.g., 1024 bits) and the padding includes the value of the length of the original message in bits. The length field is a security measure to increase the difficulty for an attacker to produce an alternative message with the same hash value.

Hash Functions & Message Authentication

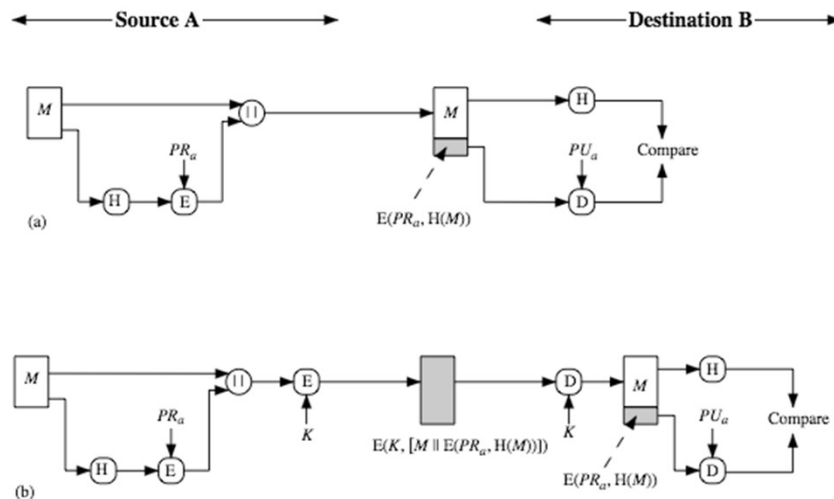


Message authentication is a mechanism or service used to verify the integrity of a message, by assuring that the data received are exactly as sent. Stallings Figure 11.2 illustrates a variety of ways in which a hash code can be used to provide message authentication, as follows:

- The message plus concatenated hash code is encrypted using symmetric encryption. Since only A and B share the secret key, the message must have come from A and has not been altered. The hash code provides the structure or redundancy required to achieve authentication.
- Only the hash code is encrypted, using symmetric encryption. This reduces the processing burden for those applications not requiring confidentiality.
- Shows the use of a hash function but no encryption for message authentication. The technique assumes that the two communicating parties share a common secret value S . A computes the hash value over the concatenation of M and S and appends the resulting hash value to M . Because B possesses S , it can recompute the hash value to verify. Because the secret value itself is not sent, an opponent cannot modify an intercepted message and cannot generate a false message.
- Confidentiality can be added to the approach of (c) by encrypting the entire message plus the hash code.

When confidentiality is not required, method (b) has an advantage over methods (a) and (d), which encrypts the entire message, in that less computation is required.

Hash Functions & Digital Signatures



Another important application, which is similar to the message authentication application, is the digital signature. The operation of the digital signature is similar to that of the MAC. In the case of the digital signature, the hash value of a message is encrypted with a user's private key. Anyone who knows the user's public key can verify the integrity of the message that is associated with the digital signature. In this case an attacker who wishes to alter the message would need to know the user's private key. As we shall see in Chapter 14, the implications of digital signatures go beyond just message authentication. Stallings Figure 11.3 illustrates, in a simplified fashion, how a hash code is used to provide a digital signature:

a. The hash code is encrypted, using public-key encryption and using the sender's private key. As with Figure 11.2b, this provides authentication. It also provides a digital signature, because only the sender could have produced the encrypted hash code. In fact, this is the essence of the digital signature technique.

b. If confidentiality as well as a digital signature is desired, then the message plus the private-key-encrypted hash code can be encrypted using a symmetric secret key. This is a common technique.

Other Hash Function Uses

- to create a one-way password file
 - store hash of password not actual password
- for intrusion detection and virus detection
 - keep & check hash of files on system
- pseudorandom function (PRF) or pseudorandom number generator (PRNG)

Hash functions are commonly used to create a one-way password file. Chapter 20 explains a scheme in which a hash of a password is stored by an operating system rather than the password itself. Thus, the actual password is not retrievable by a hacker who gains access to the password file. In simple terms, when a user enters a password, the hash of that password is compared to the stored hash value for verification. This approach to password protection is used by most operating systems.

Hash functions can be used for intrusion detection and virus detection. Store $H(F)$ for each file on a system and secure the hash values (e.g., on a CD-R that is kept secure). One can later determine if a file has been modified by recomputing $H(F)$. An intruder would need to change F without changing $H(F)$.

A cryptographic hash function can be used to construct a pseudorandom function (PRF) or a pseudorandom number generator (PRNG). A common application for a hash-based PRF is for the generation of symmetric keys. We discuss this application in Chapter 12.

Two Simple Insecure Hash Functions

- consider two simple insecure hash functions
- bit-by-bit exclusive-OR (XOR) of every block
 - $C_i = b_{i1} \text{ xor } b_{i2} \text{ xor } \dots \text{ xor } b_{im}$
 - a longitudinal redundancy check
 - reasonably effective as data integrity check
- one-bit circular shift on hash value
 - for each successive *n-bit* block
 - rotate current hash value to left by 1 bit and XOR block
 - good for data integrity but useless for security

To get some feel for the security considerations involved in cryptographic hash functions, we present two simple, insecure hash functions in this section. One of the simplest hash functions is the bit-by-bit exclusive-OR (XOR) of every block, which can be expressed as shown. This operation produces a simple parity for each bit position and is known as a longitudinal redundancy check. It is reasonably effective for random data as a data integrity check. Each *n-bit* hash value is equally likely. Thus, the probability that a data error will result in an unchanged hash value is 2^{-n} . With more predictably formatted data, the function is less effective. For example, in most normal text files, the high-order bit of each octet is always zero. So if a 128-bit hash value is used, instead of an effectiveness of 2^{-128} , the hash function on this type of data has an effectiveness of 2^{-112} .

A simple way to improve matters is to perform a one-bit circular shift, or rotation, on the hash value after each block is processed. Although this second procedure provides a good measure of data integrity, it is virtually useless for data security when an encrypted hash code is used with a plaintext message. Given a message, it is an easy matter to produce a new message that yields that hash code: Simply prepare the desired alternate message and then append an *n-bit* block that forces the new message plus block to yield the desired hash code.

Hash Function Requirements

Requirement	Description
Variable input size	H can be applied to a block of data of any size.
Fixed output size	H produces a fixed-length output.
Efficiency	H(x) is relatively easy to compute for any given x, making both hardware and software implementations practical.
Preimage resistant (one-way property)	For any given hash value h , it is computationally infeasible to find y such that $H(y) = h$.
Second preimage resistant (weak collision resistant)	For any given block x , it is computationally infeasible to find $y \neq x$ with $H(y) = H(x)$.
Collision resistant (strong collision resistant)	It is computationally infeasible to find any pair (x, y) such that $H(x) = H(y)$.
Pseudorandomness	Output of H meets standard tests for pseudorandomness

Stallings Table 11.1 lists the generally accepted requirements for a cryptographic hash function. The first three properties are requirements for the practical application of a hash function. The fourth property, preimage (for a hash value $h = H(x)$, we say that x is the **preimage** of h) resistant, is the one-way property: it is easy to generate a code given a message, but virtually impossible to generate a message given a code. This property is important if the authentication technique involves the use of a secret value (Figure 11.2c). The fifth property, second preimage resistant, guarantees that it is impossible to find an alternative message with the same hash value as a given message. This prevents forgery when an encrypted hash code is used (Figure 11.2b and Figure 11.3a). A hash function that satisfies the first five properties in Table 11.1 is referred to as a weak hash function. If the sixth property, collision resistant, is also satisfied, then it is referred to as a strong hash function. A strong hash function protects against an attack in which one party generates a message for another party to sign. The final requirement, **pseudorandomness**, has not traditionally been listed as a requirement of cryptographic hash functions, but is more or less implied.

Attacks on Hash Functions

- have brute-force attacks and cryptanalysis
- a preimage or second preimage attack
 - find y s.t. $H(y)$ equals a given hash value
- collision resistance
 - find two messages x & y with same hash so
$$H(x) = H(y)$$
- hence value $2^{m/2}$ determines strength of hash code against brute-force attacks
 - 128-bits inadequate, 160-bits suspect

As with encryption algorithms, there are two categories of attacks on hash functions: brute-force attacks and cryptanalysis. A brute-force attack does not depend on the specific algorithm but depends only on bit length. In the case of a hash function, a brute-force attack depends only on the bit length of the hash value. A cryptanalysis, in contrast, is an attack based on weaknesses in a particular cryptographic algorithm.

For a preimage or second preimage attack, an adversary wishes to find a value y such that $H(y)$ is equal to a given hash value h . The brute force method is to pick values of y at random and try each value until a collision occurs. For an m -bit hash value, the level of effort is proportional to 2^m . Specifically, the adversary would have to try, on average, 2^{m-1} values of y to find one that generates a given hash value h .

For a collision resistant attack, an adversary wishes to find two messages or data blocks, x and y , that yield the same hash function: $H(x) = H(y)$. This requires much less effort than a preimage or second preimage attack. The effort required is explained by a mathematical result referred to as the birthday paradox (next slide).

If collision resistance is required, then the value $2^{m/2}$ determines the strength of the hash code against brute-force attacks. Van Oorschot and Wiener presented a design for a \$10 million collision search machine for MD5, which has a 128-bit hash length, that could find a collision in 24 days. Thus a 128-bit code may be viewed as inadequate. The next step up, if a hash code is treated as a sequence of 32 bits, is a 160-bit hash length. With a hash length of 160 bits, the same search machine would require over four thousand years to find a collision. With today's technology, the time would be much shorter, so that 160 bits now appears suspect.

Birthday Attacks

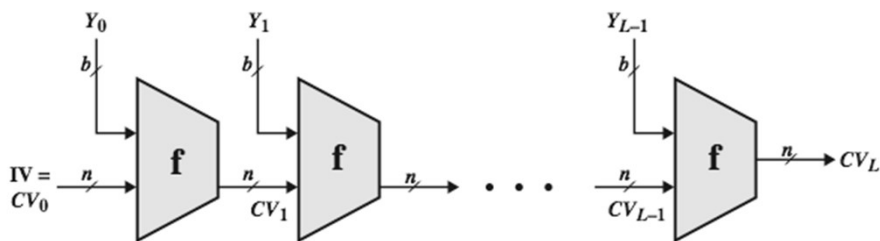
- might think a 64-bit hash is secure
- but by **Birthday Paradox** is not
- **birthday attack** works thus:
 - given user prepared to sign a valid message x
 - opponent generates $2^{m/2}$ variations x' of x , all with essentially the same meaning, and saves them
 - opponent generates $2^{m/2}$ variations y' of a desired fraudulent message y
 - two sets of messages are compared to find pair with same hash (probability > 0.5 by birthday paradox)
 - have user sign the valid message, then substitute the forgery which will have a valid signature
- conclusion is that need to use larger MAC/hash

The Birthday Attack exploits the birthday paradox – the chance that in a group of people two will share the same birthday – only 23 people are needed for a $\text{Pr} > 0.5$ of this. Can generalize the problem to one wanting a matching pair from any two sets, and show need $2^{m/2}$ in each to get a matching m -bit hash.

Yuval proposed the strategy shown to exploit the birthday paradox in a collision resistant attack. Note that creating many message variants is relatively easy, either by rewording or just varying the amount of white-space in the message. All of which indicates that larger MACs/Hashes are needed.

Hash Function Cryptanalysis

- cryptanalytic attacks exploit some property of alg so faster than exhaustive search
- hash functions use iterative structure
 - process message in blocks (incl length)
- attacks focus on collisions in function f



As with encryption algorithms, cryptanalytic attacks on hash functions seek to exploit some property of the algorithm to perform some attack other than an exhaustive search. In recent years, have much effort, and some successes, in developing cryptanalytic attacks on hash functions. Must consider the overall structure of a typical secure hash function, referred to as an iterated hash function, as indicated in Stallings Figure 11.7. This was proposed by Merkle and is the structure of most hash functions in use today. The hash function takes an input message and partitions it into L fixed-sized blocks of b bits each. If necessary, the final block is padded to b bits. The final block also includes the value of the total length of the input to the hash function. The inclusion of the length makes the job of the opponent more difficult. The hash algorithm involves repeated use of a compression function, f , that takes two inputs (an n -bit input from the previous step, called the chaining variable, and a b -bit block) and produces an n -bit output. At the start of hashing, the chaining variable has an initial value that is specified as part of the algorithm. The final value of the chaining variable is the hash value. Often, $b > n$; hence the term compression. The motivation for this iterative structure stems from the observation by Merkle and Damgard that if the compression function is collision resistant, then so is the resultant iterated hash function. Therefore, the structure can be used to produce a secure hash function to operate on a message of any length. Cryptanalysis of hash functions focuses on the internal structure of f and is based on attempts to find efficient techniques for producing collisions for a single execution of f . Once that is done, the attack must take into account the fixed value of IV . The attack on f depends on exploiting its internal structure. The attacks that have been mounted on hash functions are rather complex and beyond our scope here.

Block Ciphers as Hash Functions

- can use block ciphers as hash functions
 - using $H_0=0$ and zero-pad of final block
 - compute: $H_i = E_{M_i} [H_{i-1}]$
 - and use final block as the hash value
 - similar to CBC but without a key
- resulting hash is too small (64-bit)
 - both due to direct birthday attack
 - and to “meet-in-the-middle” attack
- other variants also susceptible to attack

A number of proposals have been made for hash functions based on using a cipher block chaining technique, but without the secret key (instead using the message blocks as keys). One of the first such proposals was that of Rabin, which divided a message M into fixed-size blocks, and used a symmetric encryption system such as DES to compute the hash code G as shown. This is similar to the CBC technique, but in this case there is no secret key. As with any hash code, this scheme is subject to the birthday attack, and if the encryption algorithm is DES and only a 64-bit hash code is produced, then the system is vulnerable. Furthermore, another version of the birthday attack can be used even if the opponent has access to only one message and its valid signature and cannot obtain multiple signings, known as a “meet-in-the-middle” attack (see text). It can be shown that some form of birthday attack will succeed against any hash scheme involving the use of cipher block chaining without a secret key provided that either the resulting hash code is small enough (e.g., 64 bits or less) or that a larger hash code can be decomposed into independent subcodes. Thus, attention has been directed at finding other approaches to hashing.

Secure Hash Algorithm

- SHA originally designed by NIST & NSA in 1993
- was revised in 1995 as SHA-1
- US standard for use with DSA signature scheme
 - standard is FIPS 180-1 1995, also Internet RFC3174
 - nb. the algorithm is SHA, the standard is SHS
- based on design of MD4 with key differences
- produces 160-bit hash values
- recent 2005 results on security of SHA-1 have raised concerns on its use in future applications

In recent years, the most widely used hash function has been the Secure Hash Algorithm (SHA). The Secure Hash Algorithm (SHA) was developed by the National Institute of Standards and Technology (NIST) and published as a federal information processing standard (FIPS 180) in 1993; a revised version was issued as FIPS 180-1 in 1995 and is generally referred to as SHA-1. The actual standards document is entitled Secure Hash Standard. SHA is based on the hash function MD4 and its design closely models MD4. SHA-1 produces a hash value of 160 bits. In 2005, a research team described an attack in which two separate messages could be found that deliver the same SHA-1 hash using 2^{69} operations, far fewer than the 2^{80} operations previously thought needed to find a collision with an SHA-1 hash [WANG05]. This result has hastened the transition to newer, longer versions of SHA.

SHA-3

- SHA-1 not yet "broken"
 - but similar to broken MD5 & SHA-0
 - so considered insecure
- SHA-2 (esp. SHA-512) seems secure
 - shares same structure and mathematical operations as predecessors so have concern
- NIST announced in 2007 a competition for the SHA-3 next gen NIST hash function

As yet, SHA-1 has not yet been "broken". That is, no one has demonstrated a technique for producing collisions in less than brute-force time. However, because SHA-1 is very similar in structure and in the basic mathematical operations used to MD5 and SHA-0, both of which have been broken, SHA-1 is considered insecure and has been phased out for SHA-2.

SHA-2, particularly the 512-bit version, would appear to provide unassailable security. However, SHA-2 shares the same structure and mathematical operations as its predecessors, and this is a cause for concern. Because it will take years to find a suitable replacement for SHA-2, should it become vulnerable, NIST decided to begin the process of developing a new hash standard. Accordingly, NIST announced in 2007 a competition to produce the next generation NIST hash function, to be called SHA-3. NIST would like to have a new standard in place by the end of 2012, but emphasizes that this is not a fixed timeline.

SHA-3

- **SHA-3 (Secure Hash Algorithm 3)** is the latest member of the Secure Hash Algorithm family of standards, released by NIST on August 5, 2015.
- SHA-3 is a subset of the broader cryptographic primitive family **Keccak**. Keccak is based on a novel approach called sponge construction.

SHA-3

- NIST does not currently plan to withdraw SHA-2 or remove it from the revised Secure Hash Standard. The purpose of SHA-3 is that it can be directly substituted for SHA-2 in current applications if necessary, and to significantly improve the robustness of NIST's overall hash algorithm toolkit.

Summary

- have considered:
 - hash functions
 - uses, requirements, security
 - hash functions based on block ciphers
 - SHA-1, SHA-2, SHA-3

Chapter 11 summary.